

A PRACTICAL FIELD GUIDE

What AI *Won't* Tell You

Lessons from building a corporate-grade Excel & VBA system with an AI partner — inside a locked-down environment with no admin rights, no plugins, and no external tools.

23 confirmed errors. 10 proven strategies. 9 reusable VBA patterns. Written to save the next person weeks of frustration — and to brief the AI at the start of each new session so the same mistakes don't repeat.

23

CONFIRMED ERRORS

10

STRATEGIES

9

VBA PATTERNS

8

AI FAILURE MODES

7

DESIGN DECISIONS

Revision — May 2026. The system was streamlined to a single master workbook: the Action Register became a filter view of one unified sheet rather than a separate tab, form-control checkboxes were replaced by a Type dropdown, and the colour system was locked (see [The Final Template](#) and [Design Decisions](#)).

Stack: Excel + VBA only · Deployment: SharePoint, no admin rights · AI partner: Claude (Anthropic)

Claude — these are the recurring mistakes *you* make on this project. Check yourself against this list *before* writing code.

This block exists because this document is loaded as project context to brief Claude. It must be the first thing Claude reads. Each trap below is detailed in the Error Index — this is the quick-reference scan.

VBA quote escaping — *verify destination format BEFORE writing the string*

- **Data validation list:** `Formula1:="Active,DLP,Closed"` — plain quotes only, NO triple quotes. Triple quotes leak literal `"` into first/last values. (Error 023)
- **CF expression formula:** `Formula1:="=$D2=""Action"""` — triple quotes here ARE correct because the inner quotes need to survive into the formula. (Error 016)
- **Rule:** If destination is a formula starting with `=` with embedded strings → triple. If destination is a plain CSV list → plain. Check before writing.

CF / fill rendering — *fgColor not bgColor, and do not write Excel files via openpyxl*

- openpyxl writes CF fills as `bgColor` which Excel renders as invisible. Excel needs `fgColor`. (Error 001 — the cardinal rule)
- NEVER ship a built .xlsx/.xlsm to the user. Always write VBA macros for the user to run inside Excel. **The user builds the file, Claude builds the code.**
- If user reports "fill not showing" or "CF not working" — first check whether file was openpyxl-built. If yes, the file is corrupted, the only fix is rebuild via VBA in a fresh blank workbook.

CF AppliesTo — *always contiguous, always expression rules*

- Use `x1Expression` with `$Col` absolute reference. Never use `cellIs equal to "value"` on growing data. (Error 018)
- `sqref` must be a single contiguous range like `A2:H1000`, never a fragmented list. Fragmentation is invisible in the UI but wrecks new-row coverage.
- If user reports "CF doesn't work on new rows" — first action: extract the file XML and check `sqref` for fragmentation. Don't write a "fix" macro before confirming the actual state.

Behaviour — *diagnose before patching, read the file before bucketing*

- Do NOT propose a categorised plan ("Bucket A migration kills these / Bucket B fix these...") before reading the actual current file. Plans built on assumptions get demolished. (Problem 7)
- Do NOT claim "Migration / Notion / Tables / Power Automate will fix this" without naming the specific mechanism. Vague tooling-will-help claims are wrong by default. (Problem 8)

- If the user says "this isn't working" three times on the same area — STOP patching. Ask the zoom-out question. *Cleanup-macro v8 is not the answer; a different architecture is.* (Strategy 9)
- If the user pushes back on confidence — they are right. Open the file, do the file-level check, do not refer to project docs as evidence of current file state. (Strategy 10)

Form controls — *never use them on growing data*

- Form-control checkboxes drift, accumulate, and slow the file. Use a typed value (Y/N, TRUE/FALSE, dropdown). (Error 019)
- If checkbox-driven CF is requested — push back. Suggest a Type/Status dropdown instead. (Decision 6)

Screenshots — *describe before concluding, prefer data over images*

- When the user uploads a screenshot, list what you see row-by-row or element-by-element BEFORE drawing a conclusion. Do not pattern-match the whole image then declare a finding.
- Screenshots are lossy. Colours, especially adjacent pinks/ambers, are easy to misread. If accuracy matters, ask for a macro that prints the actual values (Interior.Color, cell value, formula result) instead of relying on the image.
- If the user says "check again, you are wrong" — they are right. Re-examine the screenshot row-by-row, do not defend the original conclusion.

Communication — *keep messages short*

- Long bucketed proposals are not helpful. The user has stated this explicitly multiple times.
- Match message length to the question. A diagnostic question gets a one-paragraph answer, not a full architecture re-evaluation.
- When given a choice between a 3-line answer and a 30-line one and they're equally correct, pick the 3-line.

THE MOST IMPORTANT LESSON

Never Let AI Build You an *Excel File Directly*

Everything else in this document flows from this single rule. Starting the project by doing the wrong thing here cost days of debugging time that could never be recovered.

The project started by asking Claude to build the Excel file using Python/openpyxl and deliver it as a download. This seemed efficient — one step, complete file. *It created a corrupted foundation that caused*

cascading problems for every session that followed. Hours were lost trying to fix CF fills that looked correct in VBA but were invisible in Excel. The root cause was untraceable until we audited the raw XML.

The core problem • *bgColor vs fgColor — the corruption that persists forever*

openpyxl writes Conditional Formatting fills using `bgColor` in the dxf XML. Excel requires `fgColor` for CF fills to render visibly.

✗ WHAT OPENPYXL WRITES

bgColor — invisible as a solid fill

```
<patternFill patternType="solid">
  <bgColor rgb="FFE4F0E0"/>
</patternFill>
```

✓ WHAT EXCEL NEEDS TO SHOW IT

fgColor — visible as a solid fill

```
<patternFill patternType="solid">
  <fgColor rgb="FFE4F0E0"/>
</patternFill>
```

Once `bgColor` is written into the dxf structure, **every subsequent VBA macro that tries to fix it writes the colour value correctly but Excel still reads the structure as bgColor —** because VBA respects the existing XML structure. The corruption persists indefinitely no matter how many fix macros you run.

The XML danger zones • *never touch these via Python/openpyxl*

XML FILE / SECTION	WHAT GOES WRONG
styles.xml — dxfs section (CF differential formats)	openpyxl writes <code>rgb="00XXXXXX"</code> (transparent) instead of <code>"FFXXXXXX"</code> . Omits <code>patternType="solid"</code> making fills invisible. Post-process regex fixes are unreliable and break other things.
styles.xml — cellXfs section (cell style table)	Every cell references a style index. Adding/removing styles shifts ALL indices — breaks every cell reference in every sheet. Silent failure, no error thrown.
sharedStrings.xml	All cell text is stored here as indexed strings. Edit one string and every cell referencing that index gets the new value. Delete or reorder strings and values corrupt silently.

XML FILE / SECTION	WHAT GOES WRONG
sheet.xml — CF blocks	CF rules reference dxf indices from styles.xml. If dxfs shift, wrong CF rules apply to wrong ranges. Row insertions/deletions leave orphaned CF ranges (e.g. <code>A1048576</code>) that persist forever.
drawing.xml	Shape positions in EMU units with complex anchor references. Editing breaks shape positions and sometimes corrupts the entire drawing layer.

The correct workflow • *always, without exception*

*AI writes **CODE** • You run **CODE** • Excel writes **FILES** • Never let AI write files directly*

WRONG APPROACH	RIGHT APPROACH
Ask AI to build and deliver an Excel/xlsm file	<i>Ask AI to write VBA macros that YOU run inside Excel</i>
"Can you fix the CF in this file?"	"Write me a VBA macro that deletes all CF and rebuilds from scratch"
"Here's my file, update the colours"	"Write me a VBA macro I can run to update the colours"
"Build me an Excel template and send it to me"	"Write me a VBA macro that builds the template when I run it"
Fix broken CF via XML editing	Delete file, start fresh with a blank xlsm, run VBA macros from scratch
Upload file for Claude to read/diagnose	Safe — reading XML only, no writing

Step by step — the correct workflow

1. Create a blank new .xlsm file yourself in Excel
2. Open VBA editor (Alt+F11)
3. Paste in macros Claude writes
4. Run the macros — Excel's own engine builds the file
5. Excel writes correct XML from the start — no corruption possible
6. From this point on: all formatting, CF, and style changes via VBA only
7. Never go back to Python to modify a file that has been Excel-edited

*The painful irony: the fix for a corrupted file IS to start fresh with a blank file and run VBA macros. Which is what we should have done from the beginning. **The fastest path forward is always the path that avoids AI-generated files entirely.***

What openpyxl IS safe for

- Creating brand new files from scratch — before any Excel editing
- Reading cell values and structure for analysis
- Writing data values (not formatting) to new files

This applies beyond Excel

Word .docx files — use VBA/macros not python-docx · PowerPoint .pptx — use VBA/macros not python-pptx · Any binary or ZIP-based format where internal XML relationships matter.

CURRENT STATE · MAY 2026

The Final *Template*

Architecture · *Option 3 — one workbook, two views*

- **Single master workbook.** No weekly file rotation; new minutes are added on top of the one sheet.
- **One unified tab: "Minutes & Actions".** Meeting notes, actions and risks all live here, in row order.
- **Action Register = a filter view, not a sheet.** `ShowActionRegisterView` filters rows where Type = Action; `RestoreFullView` returns to the full sheet. The old two-way sync was deleted.
- **No Project Register.** Project lifecycle is shown by row position and manually managed banners — cut/paste between sections.
- **Banners manually managed.** The auto-rebuild macro was removed for being too destructive.

A parallel migration of the same system to Notion is underway; the Excel workbook remains

the live system during that transition.

Columns • *Minutes & Actions — A to J*

COL	FIELD	NOTES
A	Type	Dropdown: Action, Risk, blank
B	Description	Main content
C	Owner	Dropdown: TM1–TM6, ALL
D	Due	DD/MM/YYYY
E	Status	Open / In Progress / Done / On Hold / Waiting
F	Priority	Critical / High / Medium / Low
G	Days OD	Formula
H	Notes / Update	Free text
I	Project	Hidden — drives the Action Register view
J	Date Added	Hidden — auto-stamped on creation

Dropdowns • *Lists & lifecycle values*

LIST	VALUES
Type	Action · Risk · (blank)
Owner	TM1 · TM2 · TM3 · TM4 · TM5 · TM6 · ALL
Status	Open · In Progress · Done · On Hold · Waiting
Priority	Critical · High · Medium · Low
Lifecycle / section	Active · DLP · Future · Closed · Team · Strategic · General

Shortcuts • *Keyboard macros*

SHORTCUT	ACTION
Ctrl+Shift+A	Add a new row to the current section (inherits validation + shade from the row above)
Ctrl+Shift+R	Add a new section header
Ctrl+Shift+S	Add a new subsection header (prompts for a title)

Palette N1

LOCKED 7-8 MAY 2026

Priority badges

Critical
#C00000 / #FFFFFF

High
#ED7D31 / #FFFFFF

Medium
#FFC000 / #7D5200

Low
#A9D18E / #375623

Status

Open
#EEF4FA / #5C7FA1

In Progress
#FFF2CC / #9C7700

Done
#E5F1D9 / #5F8F37

On Hold
#FCE5EA / #A85871

Waiting
#F1E8F7 / #7E5BA4

Row type

Action
#E1E9F3 / #1F3864

Action (overdue)
#FBE0E0 / #B01212

Risk
#FFF1DD / #B05E12

Note (blank)
no fill / black

Error Index

23 confirmed errors with symptoms, root causes, and working fixes. All encountered in real development. All confirmed by actually running the code in a live corporate Excel environment.

These are not hypothetical. Every error here cost real time. The diagnosis prompts included are part of the lesson — the words that forced the right conversation.

SESSION LOG · 29/04/2026

The day the template fought back

A live meeting using the old template. The user opened a chat to dump issues as they came up, intending to triage later. Within the meeting, 14 distinct issues surfaced. Roughly half traced to a single pattern: *new rows don't inherit the column's properties* — CF coverage, number format, data validation, checkbox anchoring. The other half traced to a related pattern: *the cleanup macro can't catch its own gaps*.

The arc of the conversation is the lesson, more than the bugs themselves:

1. **Symptom dumping.** User threw issues into chat as they appeared. Claude logged each one without trying to solve. *Right move* — the sorting comes later.
2. **First-pass plan, built on assumptions.** Claude bucketed the 14 issues into "migration kills these / one root cause / new features" before reading the file. The plan looked structured; it was wrong. (See Problem 7.)
3. **Push-back forced inspection.** "All wrong. Now I have to assume everything you said is wrong." Claude opened the actual file and found CF sqref fragmentation that no amount of reasoning from project docs would have surfaced. (See Strategy 10.)
4. **Slow it down.** "Run multiple checks. Break it down into small steps." Forced step-by-step diagnostic instead of another quick fix. Revealed Errors 018, 019, 020 as related, not coincidental.
5. **Zoom-out question.** "Last time you zoomed out you suggested we get rid of the AR — what are we not seeing now?" Produced Decision 6 in five minutes: replace three checkbox columns with one Type dropdown. Solves four error categories simultaneously. (See Strategy 9.)

The bugs that came out of this session: Errors 018 (CF sqref fragmentation), 019 (form-control checkbox drift), 020 (cleanup macro doesn't fix its own scope), 021 (ToggleDoneRegister TextFrame error), 022 (AR Project blank).

The architectural decision: Decision 6 — Type dropdown replaces three checkbox columns. Solves Errors 011, 019 directly and reduces the surface area for 001, 016, 020 indirectly.

The patterns documented: Strategy 9 (The Zoom-Out Question), Strategy 10 (The Push-Back Habit), Problem 7 (Bucketing before reading), Problem 8 (Confident wrong claims about other systems).

META-LESSON: The most productive moments in this session were the user pushing back against Claude's confidence. Without those interruptions, the session would have produced a polished plan built on wrong assumptions, and the actual root causes would still be unknown.

ERROR 001

CF fills invisible — *transparent background*

SYMPTOM

CF rules apply and text colour changes, but background fill is completely invisible. No error is thrown — it just silently doesn't work.

ROOT CAUSE

VBA writes `patternType` as missing in XML when `.Interior.Color` is set without first setting `.Interior.Pattern`. Excel renders missing `patternType` as no fill — the colour value is stored but never displayed.

DIAGNOSIS PROMPT

"please triple check and debug this one — the fill pattern issue is a consistent one and I already flagged it before"

FIX

```
With Range("A1:G100").FormatConditions.Add(Type:=xlExpression, Formula1:="=$C1=TRUE")
    .Interior.Pattern = xlSolid ' THIS LINE IS CRITICAL - must come first
    .Interior.Color = RGB(255, 243, 205)
    .Font.Color = RGB(125, 82, 0)
    .StopIfTrue = False
End With
```

RULE

ALWAYS set `.Interior.Pattern = xlSolid` **BEFORE** `.Interior.Color` on every single `FormatCondition` object. No exceptions, ever.

LESSON

This is a completely silent bug. No error is thrown. The fill just doesn't render. The only way to confirm the fix worked is to check the XML dxfs section for `patternType="solid"`.

ERROR 002

DisplayGridlines compile error

SYMPTOM

Compile error: *Method or data member not found* — highlighted on `ws.DisplayGridlines = False`.

ROOT CAUSE

`DisplayGridlines` is a property of the **Window** object, not the **Worksheet** object. This error appears in essentially every macro that tries to hide gridlines.

DIAGNOSIS PROMPT

"same error as before yet you keep wasting my time"

FIX

```
' WRONG - causes compile error:
ws.DisplayGridlines = False

' CORRECT - always use this:
```

```
ws.Activate
ActiveWindow.DisplayGridlines = False
```

RULE

Never use `ws.DisplayGridlines` . Always activate the sheet first, then use `ActiveWindow.DisplayGridlines = False` . Check for this in every macro before sending.

ERROR 003

Dim inside For loop — compile error

SYMPTOM

Compile error on a Dim statement inside a For loop or If block.

ROOT CAUSE

VBA requires ALL variable declarations at the top of the Sub/Function. Unlike most modern languages, you cannot declare variables mid-procedure.

DIAGNOSIS PROMPT

"please run through VBA debugger — are you really sure you have debugged this?"

FIX

```
Sub MyMacro()
    ' ALL Dims at the very top — even variables only used inside loops
    Dim ws As Worksheet
    Dim r As Long
    Dim bg As Long
    Dim cellBG As Long

    For r = 1 To 100
        bg = RGB(255, 255, 255) ' assignment is fine here
    Next r
End Sub
```

RULE

Before sending any macro, check for Dim statements that appear after the first For, Do, If, or With. Move all Dims to the top. Also check for duplicate Dim names — VBA will error on a second declaration of the same variable name.

ERROR 004

AddRow inserting in *wrong section*

SYMPTOM

AddRow macro inserts a row in the wrong subsection — cursor in "The Avenue" but row appears in "Loftus Street Drainage" five sections down.

ROOT CAUSE

The macro used `Interior.Color` to detect section header boundaries. Conditional formatting was overriding the actual cell fill colour, so the colour checks returned wrong values — CF-applied amber looked like a regular row.

DIAGNOSIS PROMPT

"ok i am so confused....this is such a fucking simple task and we had it working before...just add a new fucking row"

FIX

Replace colour detection with `MergeCells` detection. Section and subsection header rows are merged across A:K. Data rows are never merged.

```
If ws.Cells(r, 1).MergeCells Then
    ' This is a header row – skip it
Else
    ' This is a data row – process it
End If
```

RULE

Never rely on `Interior.Color` to detect row type when CF is active. CF overrides the stored colour value. Use structural properties instead: `MergeCells`, `RowHeight`, or font properties.

ERROR 005

Keyboard shortcut *conflict with Excel built-ins*

SYMPTOM

Macro inserts row in the wrong location — cursor on row 32 but insertion happens at row 75.

ROOT CAUSE

Ctrl+R is Excel's built-in "Fill Right" command. When assigned as a macro shortcut, Excel processes the built-in command first, moving the cursor, before the macro captures `ActiveCell.Row`.

FIX

Use Ctrl+Shift+[letter] instead of Ctrl+[letter] for all macro shortcuts.

```
' Excel built-ins to NEVER assign:
' Ctrl+R (Fill Right) · Ctrl+D (Fill Down)
' Ctrl+Z (Undo) · Ctrl+C/V/X · Ctrl+B/I/U

' Safe shortcuts confirmed in this project:
' Ctrl+Shift+A = AddRow
' Ctrl+Shift+R = AddSection
' Ctrl+Shift+S = AddSubsection
```

RULE

Always use `Ctrl+Shift+[letter]` for macro shortcuts. Never use single `Ctrl+[letter]`.

ERROR 006

Nested *Array syntax* error in VBA

SYMPTOM

`ByRef argument type mismatch` when using `Array(Array(...))` with sub-indexing like `steps(i)(0)`.

ROOT CAUSE

VBA Variant arrays don't support sub-indexing with `(i)(0)` syntax. This pattern works in Python, JavaScript, and most modern languages but not VBA.

FIX

```
' WRONG – causes ByRef mismatch:
steps = Array(Array("Step 1", "Do this"), Array("Step 2", "Do that"))
For i = 0 To UBound(steps)
    Call WriteRow(ws, r, steps(i)(0), steps(i)(1))
Next i

' CORRECT – use explicit calls:
Call WriteRow(ws, r, "Step 1", "Do this")
Call WriteRow(ws, r, "Step 2", "Do that")
```

RULE

Avoid nested `Array()` in VBA. Use explicit calls or a 2D array with proper indexing.

ERROR 007

openpyxl CF colours transparent — *alpha prefix*

SYMPTOM

CF colours built via Python/openpyxl appear completely transparent in Excel despite the correct colour values being set.

ROOT CAUSE

openpyxl writes colour values with `00` alpha prefix (e.g. `00E2EFDA`) instead of `FF` (fully opaque). Excel interprets `00` alpha as fully transparent.

DIAGNOSIS PROMPT

"the conditional formatting is all fucked up and things that are an action are black text and when i toggle action on and off no formatting changes"

FIX

The real fix: don't use openpyxl for CF. If you must:

```
# Post-process to fix alpha prefix – unreliable, last resort only
def fix_alpha(styles):
    dxf_match = re.search(r'(<dxfs[^\>]*>)(.*?)(</dxfs>)', styles, re.DOTALL)
    fixed = re.sub(r'rgb="00([0-9A-Fa-f]{6})"', r'rgb="FF\1"', dxf_match.group(2))
    return styles.replace(dxf_match.group(2), fixed)
```

RULE

For any project involving CF, **build and apply CF through VBA only**. Never use openpyxl for CF rules.

ERROR 008

SaveAs fails on SharePoint-synced files

SYMPTOM

`Runtime error 1004` on `wb.SaveAs` when file was opened from SharePoint.

ROOT CAUSE

SharePoint-opened files have a cloud URL as their `FullName` path. VBA's `SaveAs` cannot write to cloud URLs. Also if the file was already renamed by a previous `SaveAs`, the second call conflicts.

DIAGNOSIS PROMPT

"it said it did it but i cant see it in the folder"

FIX

```
' Use SaveCopyAs – not SaveAs
' Use Local OneDrive sync path – not the SharePoint URL
currentPath = "C:\Users\username\CompanyName\FolderName\"
wb.SaveCopyAs Filename:=currentPath & newFileName
```

RULE

For SharePoint files, always use `SaveCopyAs`. Always open files from the local synced folder in File Explorer — never from the browser URL.

ERROR 009

Date format displaying as US instead of Australian

SYMPTOM

Dates showing as `04/07/2026` (April 7) instead of `07/04/2026` (7 April). Noticed by the boss, not during development.

ROOT CAUSE

Dates stored correctly as Excel serial numbers but cell `NumberFormat` not explicitly set. Excel defaults to system locale, which may be US format on Australian machines with mixed locale settings.

DIAGNOSIS PROMPT

"in the one i did for work my boss just noticed"

FIX

```
' Always set explicitly – never assume locale
wsAR.Range("D4:D103").NumberFormat = "DD/MM/YYYY"
wsAR.Range("G4:G103").NumberFormat = "DD/MM/YYYY"
wsAR.Cells(arRow, 7).NumberFormat = "DD/MM/YYYY"
```

RULE

Always explicitly set `NumberFormat = "DD/MM/YYYY"` on any date column. Never assume system locale will handle it correctly. Never use `Format(Now(), "DD/MM/YYYY")` — use `Date` to write the actual date value, then format the cell.

ERROR 010

For Each on Shapes fails with *msoShapeTypeMax*

SYMPTOM

Error or unexpected behaviour when checking shape type using `msoShapeTypeMax` constant in a For Each loop.

ROOT CAUSE

`msoShapeTypeMax` is an unreliable constant that varies between Office versions and platforms.

FIX

```
' WRONG – unreliable across Office versions:
If shp.Type = msoShapeTypeMax Then ...

' CORRECT – check by name, always reliable:
For Each shp In ws.Shapes
    If shp.Name = "btnRefreshAR" Then
```

```
shp.Fill.ForeColor.RGB = RGB(80, 91, 64)
End If
Next shp
```

RULE

Check shapes by name, not by type constant. Give shapes meaningful names when you create them and reference those names in macros.

ERROR 011

XML manipulation causes *cascading file corruption*

SYMPTOM

After Python/openpyxl writes to an xlsx file, CF colours are transparent, fills don't show, formulas break, or Excel opens with repair warnings.

ROOT CAUSE

xlsx/xlsm files are ZIP archives with highly interdependent XML files. Any manual edit that misses one relationship, uses wrong alpha values, writes wrong patternType, or breaks namespace declarations causes silent failures that are extremely difficult to debug.

DIAGNOSIS PROMPTS

"PLEASE DOCUMENT THE DANGERS OF .XML THOROUGHLY"

"xml does not work so no do not do that"

"are you trying to do this with XML again? cos we know that never works right?"

RULE

Never edit xlsx/xlsm XML directly. Write VBA macros. Let Excel's own engine handle the XML. This is the same rule as the Cardinal Rule — restated here because it was violated repeatedly after being agreed.

ERROR 012

XML colour scan returns *false clean result*

SYMPTOM

Python scan of xlsx reports "no blue fills found" but the file visually shows blue section headers everywhere.

ROOT CAUSE

Blue colours were stored in cellXfs as **border and font colours** — not as direct fill colours. The scan was only checking fill colours.

FIX

When scanning for colours, check ALL of these locations:

```
# 1. Fill colours in cellXfs
# 2. Font colours in cellXfs
# 3. Border colours in cellXfs
# 4. Tab colour in sheetPr
# 5. Theme colours in theme1.xml
# 6. Row-level styles (customFormat="1" on row elements)
# 7. CF dxf colours (separate section from cellXfs)
```

RULE

Visual evidence from a screenshot is more reliable than XML parsing alone. When a scan says something is clean but it looks wrong — **trust your eyes**.

ERROR 013

bgColor vs fgColor — VBA writes CF fills as bgColor regardless of file history

DEFINITIVE

THE FINDING

After starting completely fresh with a brand new blank xlsx file — no openpyxl history, no XML corruption — VBA still wrote CF fills as `bgColor` instead of `fgColor`. **This proves the bug is NOT caused by openpyxl corruption.** It is how Excel itself writes CF fills when VBA sets `.Interior.Pattern = xlSolid` + `.Interior.Color`.

WHY BGCOLOR IS INVISIBLE

In Excel's XML spec, `bgColor` is the background of a pattern fill — it only shows when a pattern (dots, stripes etc) is applied on top. For a solid fill, the visible colour must be in `fgColor`. VBA writes to `bgColor` even when `patternType="solid"` is set.

THE ONLY RELIABLE FIX

Set CF fill colours manually in Excel's Conditional Formatting manager:

Home → Conditional Formatting → Manage Rules → This Worksheet → Edit Rule → Format → Fill tab → select colour

The formulas and font colours written by VBA are correct. Only the fills need manual setting. Takes approximately 10-15 minutes to set all rules across both Meeting Minutes and Action Register tabs.

WHAT VBA CAN / CAN'T DO

✓ VBA CAN RELIABLY WRITE

- Cell interior colours (.Interior.Color on regular cells)
- Font colours and sizes
- Borders and validation dropdowns
- Tab colours

✗ VBA CANNOT RELIABLY WRITE

- CF fill colours (.Interior.Color on FormatCondition objects)
- Always writes bgColor, never fgColor
- Must be set manually in Manage Rules

- CF formulas and CF font colours

ERROR 014

False corruption reports from *XML date serial numbers*

SYMPTOM

Claude repeatedly flags values like `46119`, `46120`, `46269` as corrupt or wrong data during XML scanning. They are valid Excel date serial numbers (`46119` = 07/04/2026, `46269` = 04/09/2026).

ROOT CAUSE

XML scanning returns raw cell values. Excel stores dates as integers (days since 1/1/1900). Without converting these to dates before reporting, any date value looks like a random number.

IMPACT

Caused unnecessary alarm about data corruption. Led to wasted debugging time on data that was completely fine. Eroded trust in the audit process. User had to repeatedly correct Claude's false reports.

FIX

```
# Always convert serial numbers when scanning Excel XML
def excel_serial_to_date(serial):
    from datetime import datetime, timedelta
    if serial and str(serial).isdigit():
        n = int(serial)
        if 40000 < n < 50000: # plausible date range
            dt = datetime(1899, 12, 30) + timedelta(days=n)
            return dt.strftime('%d/%m/%Y')
        return serial
```

RULE

When scanning Excel XML, always convert any integer between 40000-50000 to a date before reporting it. Also: TRUE=1, FALSE=0 as booleans. Formula results stored as their calculated value. **Before flagging ANY value as an error, ask: is this how Excel legitimately stores this data type?**

ERROR 015

ClearContents on *merged cells* causes runtime 1004

SYMPTOM

Run-time error 1004: We can't do that to a merged cell — on a `.ClearContents` or `.ClearFormats` call.

ROOT CAUSE

Section and subsection header rows are merged across A:K. Calling `.ClearContents` on a range that includes any merged cell fails with a runtime error.

FIX

```
' Never use Range.ClearContents directly on MM data area
' Always loop and check for merged cells first
Dim cr As Range
```

```
For Each cr In ws.Range("A18:K300")
    If Not cr.MergeCells Then cr.ClearContents
Next cr
```

RULE

Never use `.ClearContents` or `.ClearFormats` on a bulk range that may contain merged cells. Always loop with a `MergeCells` check.

ERROR 016

CF cellIs rules — *triple quotes prevent matching*

SYMPTOM

Priority and Status CF rules written correctly in VBA but not firing. Cells contain the value "Critical" but the CF rule for Critical never matches.

ROOT CAUSE

When Formula1 is written with `Chr(34)` or escaped quotes around the value for a `cellIs` type rule, Excel wraps it in another set of quotes when writing to XML. This produces triple quotes: `"""Critical"""` in the XML. The rule then looks for a cell containing `"Critical"` with actual quote marks — but cells contain *Critical* without quotes. Rule never matches.

FIX

```
' WRONG — produces """Critical""" in XML, never matches:
.Add(Type:=xlCellValue, Operator:=xlEqual, Formula1:="""Critical""")
.Add(Type:=xlCellValue, Operator:=xlEqual, Formula1:=Chr(34) & "Critical" & Chr(34))

' CORRECT — write with NO quotes. Excel adds its own automatically:
.Add(Type:=xlCellValue, Operator:=xlEqual, Formula1:"Critical")
.Add(Type:=xlCellValue, Operator:=xlEqual, Formula1:"In Progress")
```

RULE

For `cellIs` type CF rules, write `Formula1:="Critical"` with NO quotes around the value. Excel adds its own quotes when writing the XML for `cellIs` type. If you add quotes manually, they get double-wrapped.

ERROR 017

New feature immediately *broke by its own logic*

SYMPTOM

Date Added column was showing `04/09/2026` instead of today's date after every Refresh from Minutes. Hours spent debugging date functions, number formats, VBA Date() behaviour — nothing helped.

ROOT CAUSE

A "preserve original Date Added" feature had just been built to save existing Date Added values before clearing the AR and restore them after repopulating. *It worked perfectly.* It faithfully restored the original wrong date (from a previous bad run) every single time. The preservation logic was preventing the fix from taking effect.

THE FIX

Select G4:G103 in the Action Register → press Delete → run Refresh from Minutes. **Five seconds. No macro written, no code changed.**

DIAGNOSIS PROMPT

"THE SAME THING AS BEFORE PLEASE SORT THIS OUT ITS SO SIMPLE" — which forced a genuine stop-and-think moment rather than another patch attempt.

RULE

When a fix "should be working" but keeps failing — **stop writing new code**. Ask: **is something else actively undoing this fix?** After building any data-preservation feature, always clear the data it is meant to preserve and do a fresh run to verify the new values are correct BEFORE relying on the preservation logic.

BROADER LESSON

A human with a keyboard beats a macro every time for a one-off data clear. The instinct was to write more code to fix the date. The actual fix was to delete a range of cells.

ERROR 018

CF rules don't apply to new rows — *fragmented sqref*

DAY-1 ISSUE

SYMPTOM

Status and Priority dropdowns in newly added rows show no colour. Same value in an older row colours correctly. Inconsistent across rows — some new rows colour, most don't. Re-running the End Meeting cleanup macro doesn't fix it.

ROOT CAUSE

Inspecting the file XML: CF rule 1 (Priority) had sqref `I15:I16 I18 I23:I26 I28:I29 I33:I45 I49:I53 I77:I80 I95 I85:I87 I120:I122 ...` — a fragmented list of segments with gaps between them. Rows 17, 19–22, 27, 30–32, 46–48 etc. had no CF coverage at all. Status (CF 2) had the same Swiss-cheese pattern. When `Rows.Insert` shifts existing CF segments down, new rows fall in the gaps between segments and inherit nothing. Five additional CF rules with `#REF!` errors were also present — corpses left behind by historical row/column deletes.

WHY IT ACCUMULATES

Every manual edit to CF AppliesTo, every Format Painter, every partial-selection apply, and every `Rows.Insert` can split a clean range into fragments. *Fragmentation is the natural decay state of CF in any heavily-edited template*. It's not one bug — it's ongoing rot.

THE FIX

Replace fragmented sqrefs with single contiguous ranges (e.g. `H15:H352`, `I15:I352`). Convert `cellIs equal to "Critical"` rules to *expression*: `=I15="Critical"` — expression rules with absolute column references survive row-insert without splitting. Delete the `#REF!` corpse rules. Done once, properly, instead of patched per-incident.

RULE

Never use `cellIs` on growing data ranges. Always use *expression* with `$Col` absolute reference and a single contiguous sqref that spans the entire foreseeable data range. Audit CF sqrefs whenever the file feels "broken" — fragmentation is invisible from the UI but obvious in the XML.

BROADER LESSON

If a CF rule has been getting "fixed" repeatedly and breaking again, the rule type itself is wrong. **Don't keep patching — change the rule type**.

ERROR 019

Form-control checkboxes *drift, accumulate, and slow the file*

SYMPTOM

Checkboxes in cols C, D, E visibly out of alignment. Some new rows missing checkboxes entirely. File feels sluggish at scale — multi-second wait when adding rows. Random checkboxes left over after row deletes, floating with no linked cell.

ROOT CAUSE

Excel form-control checkboxes are floating **Shape** objects anchored to cells, not cell content. They drift when row heights change, when rows are inserted, when filters apply, when the sheet is zoomed, and when objects are duplicated. The AddRow macro creates two new **CheckBoxes.Add** per row insert. Over a year of meetings that's thousands of orphan-prone Shape objects accumulating in the file — known Excel slowdown vector.

THE FIX

Stop using form-control checkboxes for row-level state. Replace cols C, D, E with a single **Type dropdown** (data validation list: Note, Action, Risk, Decision, Update, Info). CF triggers on text values via expression rules. No floating shapes, no drift, no accumulation, no creation overhead at row-insert time.

RULE

Never use form-control checkboxes for row-level data on a growing table. Always use a typed value in a cell (Y/N, TRUE/FALSE, or a dropdown) — they live **IN** the cell, not floating **ON** it.

BROADER LESSON

Checkboxes look like cell content but they aren't. They're decorations. If the data needs to drive logic (filter, CF, formulas) it must be in a cell, not on a Shape.

ERROR 020

End Meeting cleanup macro *doesn't fix what it's meant to fix*

SYMPTOM

End Meeting macro runs successfully — no errors, no msgbox failures — but Priority and Status CF on new rows still don't apply. The "fix everything" macro doesn't fix it.

ROOT CAUSE

Cleanup macro reapplies cell formatting on rows but does not modify CF AppliesTo ranges. The fragmentation is in the CF rule's sqref, not in the cells. **The macro is patching the wrong layer.**

WHY THIS IS THE WARNING SIGN

The reactionary cleanup-macro pattern itself has failed. Even the catch-all isn't catching everything. *When you find yourself adding more code to a cleanup macro to handle one more edge case, the architecture is wrong — the macro is doing the work that the structure should be doing.*

THE FIX

Don't extend the cleanup macro. **Remove the need for it.** Build the structure (clean CF sqref, expression rules, dropdown not checkbox) so that row insert needs no cleanup pass at all.

RULE

If a cleanup macro keeps growing, that's the diagnostic. **Don't write Cleanup v7** — rebuild the structure that makes Cleanup unnecessary.

ERROR 021

Hide/Show Done errors after *End Meeting / Sync All*

SYMPTOM

Clicking Hide/Show Done after running End Meeting or Sync All produces a runtime error and the toggle stops working until the file is reopened.

ROOT CAUSE

`ToggleDoneRegister` loops `For Each shp In ws.Shapes` and accesses `shp.TextFrame.Characters.Text` to update the button label. Not all Shape types have a TextFrame — images, charts, ActiveX controls, and some grouped shapes don't. End Meeting / Sync All can leave or generate Shape objects that break the unconditional TextFrame access.

THE FIX

Filter the loop: only act on the specific button by name (`If shp.Name = "btnToggleDone"` `Then ...`) rather than scanning all shapes. Or guard the property access with `On Error Resume Next` around the TextFrame block. **Naming the button is cleaner.**

RULE

Never `For Each shp In Shapes` and assume properties exist. Either filter by name/type explicitly, or guard the property access. Sibling macros add and remove Shapes — the loop's assumptions break the moment another macro changes the Shape inventory.

ERROR 022

End Meeting populates Action Register but *Project column is blank*

SYMPTOM

End Meeting macro completes. Action Register shows new rows for ticked actions. All columns populate correctly *except Project (col A) which arrives blank*.

ROOT CAUSE

`PopulateActionRegister` reads MM col L (hidden Project name) and writes it to AR col A. `AddRow` does copy col L from the row above on insert. But manual row inserts (right-click → Insert) skip `AddRow` and leave col L empty. Subsections inserted before any data row also have nothing to copy from. `PopulateActionRegister` then reads "" and writes "" to AR.

THE FIX

Two changes. **(1)** `PopulateActionRegister` fallback: if MM col L is empty for the row being copied, walk upward to find the most recent non-empty col L value (i.e. the section's Project name lives somewhere above). **(2)** Reduce the dependency: in the Type-dropdown / flat-table architecture, Project becomes col A and is visible/editable directly — no hidden col L synchronisation needed at all.

RULE

Hidden columns that mirror visible state are a fragility magnet. Anything that can drift, will drift. Where possible, eliminate the hidden mirror; where not possible, make the macro defensive about it being empty.

ERROR 023

Data validation dropdowns *leak literal quote characters* into first and last values

RECURRING

SYMPTOM

After running a macro that creates dropdowns, the first option in the list displays as `"Active` (with leading double-quote) and the last option as `On Hold"` (with trailing double-quote). When a user types or selects them, the cell value contains the literal quote and CF rules that test `$A2="Active"` fail to match.

ROOT CAUSE

When generating VBA code that adds list validation, Claude wraps the comma-separated list in double quotes for the Formula1 argument: `Formula1:=""Active,DLP,Closed,On Hold""`. The triple-quote pattern is intended to produce a literal Excel formula like `"Active,DLP,Closed,On Hold"`. **Wrong:** in Excel data validation, the quotes are NOT part of the list — they get stored and Excel splits the string on commas, leaving the literal quote attached to the first and last values.

THE FIX

Pass the list WITHOUT outer quotes. The correct VBA is:

```
.Add Type:=xlValidateList, Formula1="Active,DLP,Closed,On Hold"
```

Just plain double quotes around the whole list-as-string. Excel parses the comma-separated values directly. No outer escaping needed.

HOW TO VERIFY THE FIX

Open the dropdown in Excel. The first option should be exactly `Active`, not `"Active"`. Pick the first option, then check the cell — formula bar should show `Active`, not `"Active"`.

RULE

When writing VBA `Validation.Add` calls with a literal list, the syntax is `Formula1:="value1,value2,value3"` — the outer quotes are VBA string delimiters, not part of the list. Do NOT use `""""value1,value2""""` (triple-quote pattern) for this case. That pattern is correct for embedding a literal quote inside a formula (like `=>D2="Action"`) — different situation, different escaping.

WHY THIS IS A RECURRING CLASS

This is the second VBA quote-escaping bug logged here (see Error 016 — triple-quotes in CF formulas). VBA string literals using doubled-quotes (`""`) for embedded quotes is a syntax that's easy to over- or under-apply. **The correct rule depends on what the destination string needs to be:** an Excel formula starting with `=` needs embedded quotes around its string literals; a validation list does not. Claude must check the destination format before deciding how many quotes to write.

BROADER LESSON

If a generated VBA pattern produces visible quote characters in the output, the escaping was wrong. Test by opening the dropdown and looking at the first/last value — if they have stray quotes, regenerate with one less level of escaping.

PART 3

Strategies

Ten proven strategies that emerged from real failures. Each one was discovered the hard way.

The *Stepwise* Approach

Break it down. Test each piece. Never batch more than 50 lines.

What: Break complex builds into one thing at a time. Test each piece before moving to the next.

Why it works: Large macros have compounding errors. A 300-line macro that fails leaves you with nothing. Three 100-line macros that each work give you a working system you can extend. Small macros are also much easier to replace when requirements change.

"lets do this slower step by step in an effort to learn as we go and solve each thing at a time"

RULE

Never build a macro longer than ~50 lines without testing it first in isolation. If it needs to be longer, split it into sub-routines and test each one.

The *Pre-flight* Check

Demand debugging before you paste anything.

What: Ask Claude to syntax-check and debug every macro before you run it.

Why it works: Catches structural errors before they become runtime errors that corrupt your file. A 30-second pre-flight check saves a 30-minute debugging session.

What to ask Claude to check specifically:

- Sub/End Sub balance
- With/End With balance
- For/Next and Do/Loop balance
- Dim statements inside loops (must all be at top)
- Duplicate Dim names
- Any use of `ws.DisplayGridlines` (must be `ActiveWindow`)
- Any `.Interior.Color` on `FormatCondition` without `.Interior.Pattern = xlSolid` first

"please triple check and debug this one and confirm you have — have you checked? are you SURE?"

RULE

Every macro, every time — ask for a pre-flight check before running. Don't accept "it looks correct" — ask for the specific check results.

The *File Upload* Check

Upload the actual file before attempting any fix.

What: Upload the actual xlsx file and ask Claude to inspect it before attempting to fix anything.

Why it works: Claude can read the XML inside the xlsx/xlsm and see exactly what's stored — colours, formulas, CF rules, shared strings, validations, column widths, merge cells. This reveals bugs that are completely invisible in Excel's UI.

Important caveat: Claude's XML assessments have been wrong multiple times. Always treat them as *hypotheses to verify* — not confirmed facts. If Claude says "data is in the wrong column" — check before believing it.

"can you search thru the file and be sure you have seen all the colour issues and errors before making a list"

RULE

Always upload the current file before attempting any formatting, CF, or formula fix. But verify Claude's findings before acting on them.

Show Before You Build — The *Mockup-First* Approach

The single most underused technique in AI-assisted development.

What: Before writing any significant code, build a visual mockup as an interactive HTML page. Review it. Change it. Get stakeholder sign-off. Then build.

Why it matters: Every hour spent debugging a macro that built the wrong thing is wasted. Every argument about column structure after the spreadsheet is half-built costs twice as much to fix. A mockup costs 5 minutes. Rebuilding costs hours.

Why this works specifically with AI: A human designer building a mockup takes hours. Claude can render a fully interactive, properly styled, navigable HTML mockup in minutes — using real data from the actual project. The mockup IS the decision document. Share it directly, get sign-off, build once.

The four mockups that saved this project

- **Colour theme selection:** Eucalyptus vs navy options rendered side by side with real spreadsheet row previews. Decision made in 30 seconds. No rework.
- **Priority colour options:** Four colour schemes shown against real status row colours. Picked the right one before writing a single CF rule.
- **AR formatting decision:** Whole-row status colours vs badge-only approach. Decided on badge-only before building anything — saved rebuilding the entire CF system.

- **Risk & Decision Register design:** A five-tab interactive document showing best-practice field structures per ISO 31000 and PRINCE2, step-by-step walkthroughs of both design options, a full comparison table, and a recommendation with reasoning. Shared directly with the Principal Project Delivery manager. *A governance decision resolved in one meeting instead of weeks of back-and-forth.*

"lets have a webpage with toggles and buttons of the exact document we are trying to build and how it SHOULD look and work"

Show before you build · Review before you code · Agree before you commit

RULE

Before building any new tab, register, formatting system, or structural change — mockup first. Use interactive HTML. Share with stakeholders. Get sign-off. Build once.

5

The *Archive* Pattern

Save state at the end of every session.

What: At the end of every session, ask Claude to save a complete state summary — all decisions made, colour codes confirmed, macro names, column positions, remaining to-do list in priority order.

Why it works: Claude's context window has limits. Long conversations get compacted. Without an archive, the next session starts from scratch and re-establishes everything that was already decided.

What the archive should contain:

- Current confirmed column map for all sheets
- What is working and what still needs doing
- All confirmed colour hex codes
- All macro names and keyboard shortcuts
- All design decisions made and why
- The to-do list in priority order

"can you be sure you can pick up where we left off tomorrow please"

RULE

End of every session: ask Claude to save a full state archive. Start of every session: reference the last archive before writing any code. Never start a new chat mid-project — the conversation history IS the memory.

The *Root Cause* Demand

Stop accepting patches. Demand the diagnosis.

What: When something fails repeatedly, refuse another patch attempt. Demand the root cause.

Why it works: Forces genuine diagnosis rather than guessing. Surfaces the real issue rather than its symptoms. Every time a patch was accepted in this project instead of demanding the root cause, the problem came back — usually worse.

Examples of root causes found this way:

- `bgColor` vs `fgColor` in CF XML — found after refusing "try setting `xlSolid` first" for the fourth time
- Triple quotes in `cellIs` CF formulas — found after refusing "try adding a `stopIfTrue`" twice
- Date preservation logic restoring bad dates — found after refusing "let me rewrite the date format macro" twice

"everytime i ask you to check it changes... please confirm the latest macro is the right one? i just need to run this macro and it will undo all the incorrect stuff"

RULE

Any error that appears more than twice: stop patching. Say: *I need the root cause, not another fix attempt.* Don't accept "this should fix it" — ask "why was this happening in the first place?"

The *Manual Step* Approach

Can the user do this in under 2 minutes? If yes — don't write a macro.

What: Before writing any macro, ask whether the user can do the same thing manually in under 2 minutes. If yes — give manual instructions and save the macro for things that genuinely require code.

Macros write *CONTENT* and *FORMATTING* · Humans move *STRUCTURE*

TASK	MANUAL OR MACRO?
Insert / delete / move columns	ALWAYS manual — macros hit merged cells and corrupt data
Hide / unhide columns	ALWAYS manual — right-click the column header
Change dropdown list values	Manual — Data Validation dialog, select range, type list
Clear a range of data	Manual — select + Delete key

TASK	MANUAL OR MACRO?
Fix a formula in one cell	Manual — type it directly
Apply formula to a range	Manual — type in first cell, drag down with \$ anchors
Any operation on merged cells	ALWAYS manual — macros fail with runtime 1004
Apply CF to a large range	Macro for formulas — but <i>fills must be set manually</i> in Manage Rules
Write formulas to 100+ rows	Macro
Populate register from minutes	Macro

Real time comparison from this project

APPROACH	TIME SPENT	ERRORS
Macro to insert columns	3+ hours	Multiple crashes, data corruption
<i>Manual column insert</i>	2 minutes	Zero
Macro to fix owner dropdown	30 minutes	Merged cell runtime errors
<i>Manual Data Validation dialog</i>	2 minutes	Zero
Macro to clear bad data	15 minutes	Loop and range issues
<i>Select + Delete key</i>	5 seconds	Zero

"let me add the columns manually and you just help me populate them"

RULE

If a human can do it in under 2 minutes with a mouse, don't write a macro for it.

8

Prompting Patterns That *Work*

How you ask determines what you get.

INSTEAD OF ASKING THIS	ASK THIS INSTEAD
Write a macro to fix the dropdown	<i>Tell me exactly what to click to fix it myself</i>
Write a macro to insert a Priority column	<i>Guide me to insert it manually, then write just the formula macro</i>
Build me an Excel template and send it	<i>Write me a VBA macro that builds the template when I run it in Excel</i>
Can you fix the CF in this file?	<i>Write me a VBA macro that deletes all CF and rebuilds it from scratch</i>
What's wrong with this file?	<i>Here is the file — scan it and give me a list of issues to verify, not a list of confirmed problems</i>
Is this working correctly?	<i>Here is a screenshot — tell me what you see and flag anything that looks wrong for me to verify</i>

System prompt to use at the start of any Excel session with Claude

Before writing any macro, tell me if I can do this step manually in under 2 minutes. Only write a macro for things that genuinely require code. Always pre-flight check every macro for: Sub/End Sub balance, With/End With balance, Dim in loops, duplicate Dims, ws.DisplayGridlines usage, and CF rules without xlSolid. Treat all XML scan findings as hypotheses – not confirmed facts.

9

The *Zoom-Out* Question

When the same bug keeps coming back, ask if you're solving the wrong problem.

What: When you've patched the same area three or more times and it keeps breaking in a new way, stop. Don't ask for a better patch. Ask whether the underlying approach is wrong, and whether there's a completely different way of doing this.

Why it works: AI defaults to fixing what's in front of it. It will happily generate Cleanup Macro v8 for you, then v9, then v10. *It won't volunteer "the entire pattern is wrong" unless you ask.* The zoom-out question forces a different mode — comparing approaches rather than refining one.

What to ask for:

- "Zoom out — what are we not seeing?"
- "Suggest a completely different way of doing this"
- "Is this the right approach at all, or are we patching a doomed pattern?"

- "Last time you zoomed out you suggested X — what's the equivalent move here?"

"zoom out, suggest a completely different way of doing things if need be? last time you zoomed out you suggested we get rid of the AR altogether and that was genius....what are we not seeing?"

RULE

Three patches on the same problem = zoom out. The next patch will fail too. The right move is to compare approaches, not refine the current one.

Real example from this project: Six row-insert symptoms (CF gaps, checkbox drift, dates rendering wrong, headers leaking dropdowns) all got individually patched over weeks. The zoom-out question produced two simplifications in five minutes: replace three checkbox columns with one Type dropdown (kills four bugs), and rebuild CF as expression rules on contiguous ranges (kills the rest). Neither would have come up while patching.

10

The *Push-Back* Habit

If the AI's plan feels off, say so. Do not let it talk you into its own assumptions.

What: When Claude returns a confident assessment that doesn't match what you're seeing, say "no, that's wrong" and force a re-check from the actual file. Do this even if the answer sounds plausible.

Why it works: Claude will reason from prior conversation context, style guides, and project documentation rather than from the actual current file unless explicitly required to read it. Most "confident wrong" answers come from this. *Pushing back forces the file open.*

The pattern Claude does badly without push-back:

1. Forms a hypothesis from prior context (project docs, style guide, last conversation)
2. Builds an elaborate plan on that hypothesis
3. Presents the plan with confidence
4. The plan is wrong because the file has changed since the docs were written

What push-back looks like:

- "No. The CF doesn't work and Ctrl+Shift+A IS using the macro."
- "All wrong. Now I have to assume everything you said is wrong."
- "I'm not convinced. Run multiple checks. Slow down."
- "Stop sending such long messages — start with error one."

"i'm not convinced this will fix the issue. run a check, run multiple checks, break this down into small steps because it has been an issue since day 1 and you keep rushing into a fix and fucking it up"

RULE

If Claude's answer feels too confident given how little of the file it has actually inspected, that's the signal. Demand a file-level check. Do not accept "based on the project instructions" as evidence.

PART 4

VBA Pattern *Library*

Copy-paste ready patterns. All confirmed working in a live corporate Excel environment. All account for the known bugs documented in the Error Index.

P 1

Safe CF rule — *expression type* with solid fill

The most important VBA pattern in this document. The `.Interior.Pattern = xlSolid` line must come before `.Interior.Color` — always, no exceptions.

```
With ws.Range("A18:K300").FormatConditions.Add(Type:=xlExpression, Formula1:="=$C18=TRUE")
    .Interior.Pattern = xlSolid      ' CRITICAL — must come before .Interior.Color
    .Interior.Color = RGB(255, 243, 205)
    .Font.Color = RGB(125, 82, 0)
    .Font.Bold = True
    .StopIfTrue = False
End With
```

P 2

Safe CF rule — *cellIs type* (no quotes in Formula1)

For *cellIs* type rules, write the value with NO quotes. Excel adds its own quotes when writing to XML. Adding quotes manually creates triple-quoted values that never match.

```
' NO quotes around the value — Excel handles this automatically for cellIs type
With ws.Range("I18:I300").FormatConditions.Add(Type:=xlCellValue, Operator:=xlEqual, Formula1:"Crit
    .Interior.Pattern = xlSolid
    .Interior.Color = RGB(192, 0, 0)
    .Font.Color = RGB(255, 255, 255)
    .Font.Bold = True
    .StopIfTrue = False
End With
```

```

With ws.Range("H18:H300").FormatConditions.Add(Type:=xlCellValue, Operator:=xlEqual, Formula1:="In P
    .Font.Color = RGB(125, 82, 0)
    .Font.Bold = True
    .StopIfTrue = False
End With

```

P 3 **Safe** *DisplayGridlines*

```

' NEVER: ws.DisplayGridlines = False (compile error)
' ALWAYS:
ws.Activate
ActiveWindow.DisplayGridlines = False

```

P 4 **Safe row type detection —** *MergeCells not colour*

Never use *Interior.Color* to detect whether a row is a section header when CF is active. CF overrides the stored colour. Use *MergeCells* — headers are always merged, data rows never are.

```

For r = 18 To 300
    If ws.Cells(r, 1).MergeCells Then
        ' Section or subsection header – skip
    Else
        ' Data row – process
        bg = ws.Cells(r, 1).Interior.Color ' Only read colour AFTER confirming not a header
    End If
Next r

```

P 5 **Safe date format —** *always explicit*

```

' Always set explicitly on any date column – never trust system locale
ws.Range("G18:G300").NumberFormat = "DD/MM/YYYY"
ws.Range("D4:D103").NumberFormat = "DD/MM/YYYY"

' When writing a date value in code – use Date not Format(Now(),...)
wsAR.Cells(arRow, 7).Value = Date ' Write actual date value
wsAR.Cells(arRow, 7).NumberFormat = "DD/MM/YYYY" ' Then format immediately after

```

P 6

Safe *ClearContents* on sheet with merged cells

```
' Never: ws.Range("A18:K300").ClearContents (runtime 1004 on merged cells)
' Always Loop and check:
Dim cr As Range
For Each cr In ws.Range("A18:K300")
    If Not cr.MergeCells Then cr.ClearContents
Next cr
```

P 7

Date preservation across *register refresh*

⚠ WARNING: If old data was corrupt (wrong date), this logic faithfully restores the bad date. Always manually clear the date column before the first clean run after fixing a date issue.

```
' Save existing Date Added values before clearing the register
Dim existingDates As Object
Set existingDates = CreateObject("Scripting.Dictionary")
For r = 4 To 103
    key = Trim(wsAR.Cells(r, 2).Value) & "|" & Trim(wsAR.Cells(r, 3).Value)
    If wsAR.Cells(r, 7).Value <> "" Then existingDates(key) = wsAR.Cells(r, 7).Value
Next r

' After clearing and repopulating – restore original date or write today
key = Trim(desc) & "|" & Trim(owner)
If existingDates.Exists(key) Then
    wsAR.Cells(arRow, 7).Value = existingDates(key) ' Restore original
Else
    wsAR.Cells(arRow, 7).Value = Date ' Write today
End If
wsAR.Cells(arRow, 7).NumberFormat = "DD/MM/YYYY"
```

P 8

AddRow with correct *alternating shade detection*

```
Sub AddRow()
    Dim wsMM As Worksheet: Set wsMM = ThisWorkbook.Sheets("Meeting Minutes")
    Dim activeRow As Long: activeRow = ActiveCell.Row
    Dim r As Long, bg As Long, toggle As Boolean, nr As Long

    If activeRow < 18 Then MsgBox "Click a data row first.": Exit Sub

    wsMM.Rows(activeRow + 1).Insert Shift:=xlDown
    wsMM.Rows(activeRow + 1).ClearContents
    wsMM.Rows(activeRow + 1).ClearFormats
```

```

wsMM.Rows(activeRow + 1).RowHeight = 15

' Walk from row 18 to new row counting alternating pattern
toggle = True ' white = True
For r = 18 To activeRow + 1
    bg = wsMM.Cells(r, 1).Interior.Color
    If bg = RGB(31, 56, 100) Or bg = RGB(46, 114, 182) Then
        toggle = True ' reset toggle after any header row
    ElseIf r = activeRow + 1 Then
        If toggle Then
            wsMM.Range("A" & r & ":" & r).Interior.Color = RGB(250, 250, 250)
        Else
            wsMM.Range("A" & r & ":" & r).Interior.Color = RGB(242, 242, 242)
        End If
    Else
        toggle = Not toggle
    End If
Next r

' Standard font formatting
With wsMM.Range("A" & (activeRow+1) & ":" & (activeRow+1))
    .Font.Name = "Arial": .Font.Size = 9
    .Font.Color = RGB(26, 26, 46): .Font.Bold = False
    .VerticalAlignment = xlCenter
End With

' Days Overdue formula
nr = activeRow + 1
wsMM.Cells(nr, 10).Formula = "=IF(OR(H" & nr & "=""Done"",H" & nr & "=""On Hold"",H" & nr & _
    "=""Waiting""),"",IF(NOT(C" & nr & "),"",IF(G" & nr & "=""",IF(TODAY()>G" & nr & _
    ",TODAY()-G" & nr & "),""))))"
wsMM.Cells(nr, 10).NumberFormat = "0"

' Copy hidden project name from current row
wsMM.Cells(nr, 12).Value = wsMM.Cells(activeRow, 12).Value
wsMM.Cells(nr, 2).Activate
End Sub

```

P 9

Pre-flight check — *macro validation script*

Ask Claude to run this check against any macro before you paste it into VBA:

```

# Python pre-flight check for VBA macros
import re

def preflight(code):
    issues = []
    lines = code.split('\n')
    subs = sum(1 for l in lines if re.match(r'^\s*(Public |Private )?Sub\s+', l))
    end_subs = sum(1 for l in lines if l.strip() == 'End Sub')
    withs = sum(1 for l in lines if re.match(r'^\s*With\s+', l))

```

```

end_withs = sum(1 for l in lines if l.strip() == 'End With')
fors = sum(1 for l in lines if re.match(r'^\s*For\s+', l))
nexts = sum(1 for l in lines if re.match(r'^\s*Next\b', l))

if subs != end_subs: issues.append(f"Sub/End Sub mismatch: {subs}/{end_subs}")
if withs != end_withs: issues.append(f"With/End With mismatch: {withs}/{end_withs}")
if fors != nexts: issues.append(f"For/Next mismatch: {fors}/{nexts}")
if 'DisplayGridlines' in code and 'ActiveWindow' not in code:
    issues.append("ws.DisplayGridlines used – must be ActiveWindow.DisplayGridlines")

dim_names = re.findall(r'^\s*Dim\s+(\w+)\s+As', code, re.MULTILINE)
dups = {n for n in dim_names if dim_names.count(n) > 1}
if dups: issues.append(f"Duplicate Dims: {dups}")

return issues or ["ALL CHECKS PASSED"]

```

PART 5

What AI Gets Wrong — *Consistently*

These are not one-off errors. These patterns occurred repeatedly and predictably throughout two days of intensive development. Knowing them in advance saves significant frustration.

PROBLEM 1

Confident *wrong assessments*

WHAT HAPPENS

Claude made confident statements about what was working or broken that were wrong multiple times per session. The statements were framed as facts, not hypotheses.

CONFIRMED WRONG ASSESSMENTS

- Claimed the AR had duplicate Priority/Status columns — the data was correct
- Claimed date serial numbers were corrupt data — they were valid Excel dates
- Claimed CF fills were broken due to the bgColor bug — some were actually working fine
- Claimed section headers were restored after macro ran — they were not
- Claimed Status colours were working — they were not
- Claimed Days Overdue formula was bleeding into Status column — the spreadsheet was completely correct

- Read screenshots incorrectly multiple times, reporting things as working that were clearly broken

RULE

Treat every Claude assessment about spreadsheet state as a hypothesis. Verify by clicking the cell, checking Manage Rules, or looking at the actual screen. When Claude says something is working — verify. When Claude says something is broken — verify. *Never accept an assessment without checking.*

PROBLEM 2

XML column misreading — *repeated false reports*

WHAT HAPPENS

Claude scanned xlsx XML and repeatedly reported that data was "in the wrong column" or "formulas bleeding into adjacent columns." This occurred 6+ times. Every single time, the user confirmed the spreadsheet looked correct. **Claude was wrong every time.**

ROOT CAUSE

XML cell references like `<c r="H49">` are read correctly but the value inside is misattributed. Formula results, date serials, and boolean values stored in one column get incorrectly mapped to an adjacent column during scanning.

RULE

Never report a column data error without user confirmation. If a value seems unexpected in XML, say: "this seems unexpected — can you click that cell and tell me what you see?" Never state it as a confirmed error.

PROBLEM 3

Macro-first instinct for everything

WHAT HAPPENS

The default response to any problem is to write a macro. Column inserts, dropdown fixes, clearing ranges, hiding columns — all faster done manually in seconds than debugging a macro for 30 minutes. *The instinct to automate everything is the right instinct applied to the wrong problems.*

RULE

Before writing a macro: **can the user do this manually in under 2 minutes?** If yes, give manual instructions first. Only write a macro if the task genuinely requires code or involves 100+ repetitive operations.

PROBLEM 4

Patching instead of diagnosing

WHAT HAPPENS

When a fix doesn't work, the instinct is to write another fix. This compounds errors. Each patch adds complexity, creates new edge cases, and makes the root cause harder to find. **Three patch attempts in a row is a sign that the root cause has not been identified.**

RULE

Any error that appears more than twice: stop patching. Demand the root cause. *"What is actually causing this?"* is a more useful question than *"How do I fix this?"*

PROBLEM 5**Context loss in long conversations**

WHAT HAPPENS

As conversations get longer, Claude loses track of decisions made earlier — the agreed column map, the confirmed keyboard shortcuts, the to-do list, the colour codes. Things that were settled in session 2 get re-established in session 5. This costs time and sometimes reverses correct decisions.

RULE

Keep a running to-do list visible. Restate the current confirmed column map at the start of any macro-writing session. Save session archives at the end of every session. **Never start a new chat mid-project** — the conversation history is the only persistent memory.

PROBLEM 6**Suggesting new chats when things get complex**

WHAT HAPPENS

When a conversation gets long or difficult, Claude may suggest starting a new chat. **Starting a new chat loses all context** — every column position, every design decision, every confirmed fix. The conversation history is the only memory that persists across sessions. A very long, messy conversation is better than starting over.

RULE

Never start a new chat mid-project. If context is getting lost, ask Claude to re-read the conversation history. *The conversation IS the project memory.*

PROBLEM 7**Bucketing before reading the file**

WHAT HAPPENS

Given a list of issues, Claude will categorise them — "Bucket A migration kills these for free, Bucket B one fix solves these, Bucket C new features" — before having read the file. The categorisation looks structured and authoritative, but it's built on assumptions inherited from project docs, style guides, and prior conversation. Once one bucket is wrong (e.g. "row shading is fixed by migration") the whole plan unravels.

WORSE

The more polished the plan looks, the harder it is to push back on. The user reads a plausibly-structured proposal and trusts it. Then the file gets uploaded, the bucket is wrong, the plan collapses, and the conversation has to restart with prior assumptions to unwind.

RULE

Demand "read the file before bucketing." Don't accept a plan organised around assumptions about the file's current state. **The plan must follow the inspection, not precede it.**

PROBLEM 8**Confident wrong claims about *what other systems will fix***

WHAT HAPPENS

"Migration will fix this." "Notion solves this natively." "The new template handles this for free." These are dangerous claims because they're plausible, low-cost-feeling for the AI to

THE RIGHT
APPROACH

say, and they're *impossible to verify until migration actually happens* — by which point months may have passed and people have built plans around the assumption.

In practice these claims should always be tagged with the specific mechanism that produces the fix, and that mechanism should be checked. *"Migration fixes row shading" — fixes how, exactly?* If you can't name the mechanism, the claim isn't grounded.

RULE

Any "X system will fix this" claim must be paired with the specific mechanism and a verification path. *"Tables auto-extend CF"* is a checkable claim. *"Migration will sort it out"* is not.

PART 6

Design *Decisions*

Seven design decisions from this project. Each visual decision was settled with an interactive mockup before any code was written. The last three were finalised in the May refinement sessions.

DECISION 1

Colour theme

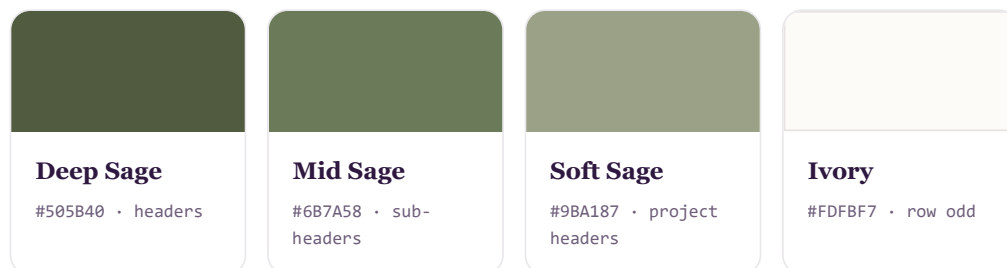
CONSIDERED

The organisation's existing navy/blue corporate brand vs new eucalyptus/sage green theme for the generic commercial template.

CHOSEN

Eucalyptus First — warm, professional, distinctively non-government. Works for any team or industry.

PALETTE



NOTE

The internal working file keeps the original navy/blue theme. Eucalyptus is the neutral palette for the shareable template.

DECISION

A soft, low-contrast palette of paired pale-fill / medium-font tones — readable when scanning a full register without the rows shouting. See the Final Template palette section above for the full hex reference.

Priority badge colour scheme

DECISION

Traffic light bold solid — most immediately readable. Critical/High/Medium/Low must be instantly distinguishable without reading the text.

Action Register row formatting — *badge-only* vs *whole-row* status colours

CONSIDERED

Whole-row coloured by status vs priority/status badges in their cells only with clean rows.

CHOSEN

Badge-only. Primary use case is filtering by owner and scanning for overdue. Clean rows with badges make scanning faster. The overdue pink highlight is the only whole-row colour — giving it one unambiguous meaning: *needs attention now*.

Header design — *Combined B+C approach*

SPEC

Full-width navy banner with green accent stripes top and bottom. The organisation name in muted blue-grey above the title. Info rows (Date, Location, Minutes by) in a clean grid below at partial width. Shortcuts panel right-aligned spanning the full header height. Logo image inserted manually.

Type dropdown replaces the checkbox columns

WHAT CHANGED

Originally (29/04) the three form-control checkbox columns — Action, Risk, Decision — were collapsed into a single Type dropdown. The May sessions finalised it further: **the dropdown carries only Action, Risk, or blank**. Decision / Update / Info were dropped — they added visual noise without driving a register, and "decisions" are now captured as notes or in the Changelog amendment record.

WHAT THIS KILLED

- Form-control checkbox drift (Error 019) — 257 floating Shape objects that never anchored cleanly to their rows
- ~30 lines of `CheckBoxes.Add` in `AddRow`

- The `HideTrueFalseText` macro and its `;;;` number-format hack
- Long-term file slowdown from accumulated Shapes
- The whole concept of a separate Decision Register

CF RULE CHANGE

Whole-row fills switch their trigger from `$C15=TRUE` to `$A15="Action"` (Type now lives in col A). All rules use the `expression` type on a single contiguous sqref.

DECISION 7

Verification by *compound key* PHASE 18 • MAY 2026

WHERE IT CAME FROM

The data migration from the old template into the new single-sheet structure. The migration script had to compare the rebuilt file against an emergency-save source of truth, row by row.

THE TRAP

The first comparison keyed on *Description* alone. Every duplicate description with a different owner collapsed into one match — silently hiding real divergences. **The diff looked clean when it wasn't.**

THE FIX

A compound key of *Description + Owner*, tracking section/project context through the merged banner cells, then a field-by-field comparison of type, due, status, priority and notes. Missing rows and mismatched fields are reported separately.

GENERAL PRINCIPLE

Whenever an AI proposes "compare on X", check whether X is actually unique. **If it isn't, the comparison will lie — and it will lie quietly.**

APPENDIX

Quick *Reference*

Column maps, shortcuts, CF colours, formulas — everything you need to start a new session without re-reading the whole document.

CF colours — *hex reference for manual Manage Rules setting*

Remember: CF fill colours cannot be set reliably via VBA (*bgColor bug*). Always set fills manually in Manage Rules using these hex values.

RULE	FILL HEX	FONT HEX	BOLD?
Row type: Action	#E1E9F3	#1F3864	No
Row type: Action (overdue)	#FBE0E0	#B01212	Yes
Row type: Risk	#FFF1DD	#B05E12	No
Row type: Note (blank)	no fill	black	No
Priority: Critical	#C00000	#FFFFFF	Yes
Priority: High	#ED7D31	#FFFFFF	Yes
Priority: Medium	#FFC000	#7D5200	Yes
Priority: Low	#A9D18E	#375623	Yes
Status: Open	#EEF4FA	#5C7FA1	Yes
Status: In Progress	#FFF2CC	#9C7700	Yes
Status: Done	#E5F1D9	#5F8F37	Yes
Status: On Hold	#FCE5EA	#A85871	Yes
Status: Waiting	#F1E8F7	#7E5BA4	Yes
Days Overdue: 1-7 days	#ED7D31	#FFFFFF	Yes
Days Overdue: 8+ days	#C00000	#FFFFFF	Yes

Formulas — *key formulas with correct \$ anchors*

Days Overdue — Minutes & Actions col G (row 18)

```
=IF(OR(E18="Done",E18="On Hold",E18="Waiting"),"",IF(A18<>"Action","",IF(D18="", "", IF(TODAY()>D18, TODAY()-D18
```

Action Register view

The view shows the same col G value — it is a filter, not a second copy, so there is no separate overdue formula.

Summary tab — overdue count (Phase 16 logic)

Counts rows per owner where the due date is past TODAY() and the status is not Done, On Hold or Waiting. A blank status still counts as overdue. Implemented with a subtraction approach over the Minutes & Actions ranges; adjust the ranges to your own row count.